

23 | OpenClip: 让我们搞清楚图片说了些什么

2023-04-28 徐文浩 来自北京

《AI大模型之美》



你好，我是徐文浩。

前面我们已经学完了文本和音频的部分。接下来，我们就要进入课程的最后一部分，也就是图像模块了。

与视觉和语音一样，Transformer 架构的模型在过去几年里也逐渐成为了图像领域的一个主流研究方向。自然，发表了 GPT 和 Whisper 的 OpenAI 也不会落后。一贯相信“大力出奇迹”的 OpenAI，就拿 4 亿张互联网上找到的图片，以及图片对应的 ALT 文字训练了一个叫做 CLIP 的多模态模型。今天，我们就看看在实际的应用里怎么使用这个模型。在学习的过程中你会发现，**我们不仅可以把它拿来做常见的图片分类、目标检测，也能够用来优化业务场景里面的商品搜索和内容推荐。**

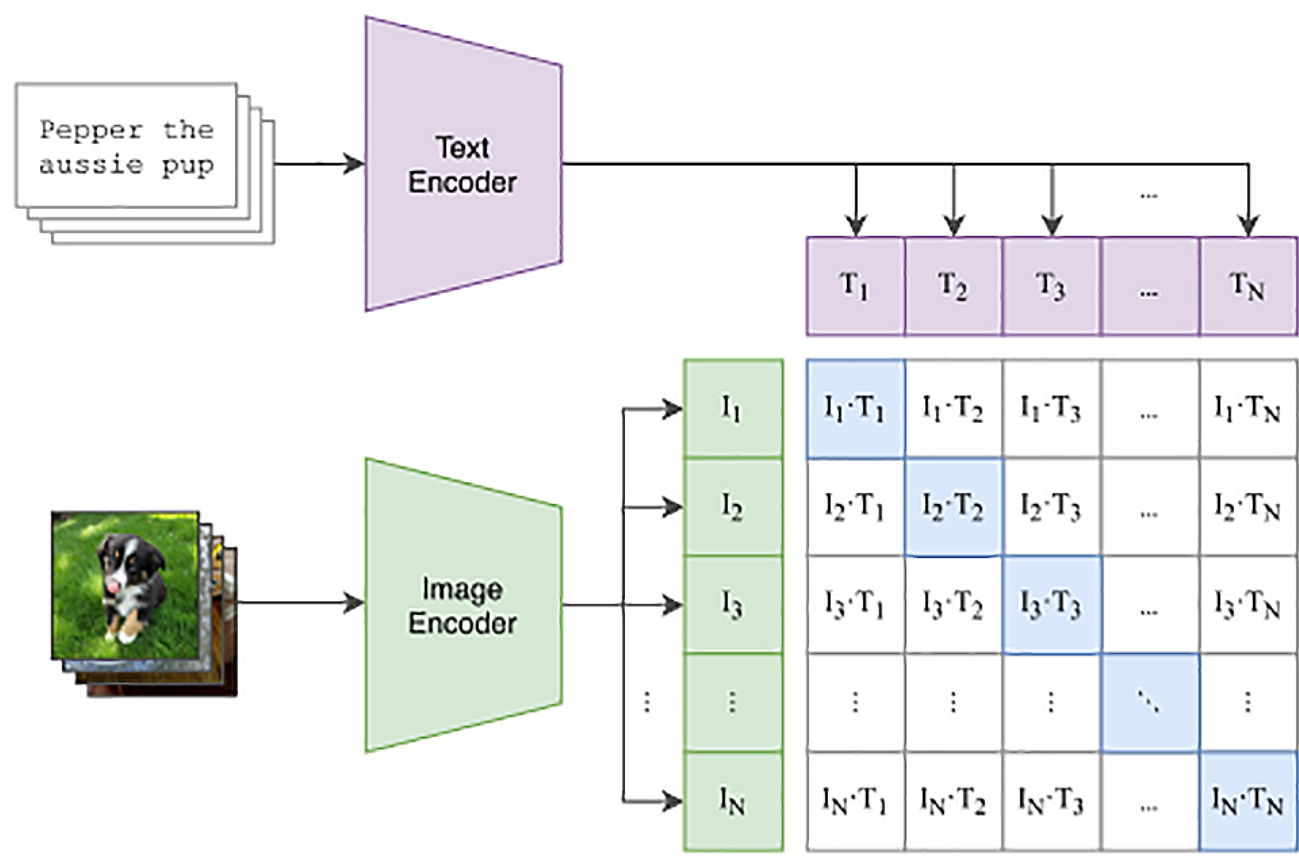
多模态的 CLIP 模型

相信你最近已经听到过很多次“多模态”这个词儿了，无论是在 OpenAI 对 GPT-4 的介绍里，还是我们在之前介绍 llama-index 的时候，这个名词都已经出现过了。

所谓“多模态”，就是多种媒体形式的内容。我们看到很多评测里面都拿 GPT 模型来做数学试题，那么如果我们遇到一个平面几何题的话，光有题目的文字信息是不够的，还需要把对应的图形一并提供给 AI 才可以。而这也是我们通往通用人工智能的必经之路，因为真实世界就是多模态的。我们每天除了处理文本信息，还会看视频、图片以及和人说话。

而 CLIP 这个模型，就是一个多模态模型。一如即往，OpenAI 仍然是通过海量数据来训练一个大模型。整个模型使用了互联网上的 4 亿张图片，它不仅能够分别理解图片和文本，还通过对比学习建立了图片和文本之间的关系。这个也是未来我们能够通过写几个提示词就能用 AI 画图的一个起点。

(1) Contrastive pre-training



图片来自<http://proceedings.mlr.press/v139/radford21a/radford21a.pdf>

CLIP 的思路其实不复杂，就是互联网上已有的大量公开的图片数据。而且其中有很多已经通过 HTML 标签里面的 title 或者 alt 字段，提供了对图片的文本描述。那我们只要训练一个模型，将文本转换成一个向量，也将图片转换成一个向量。图片向量应该和自己的文本描述向量的距离尽量近，和其他的文本向量要尽量远。那么这个模型，就能够把图片和文本映射到同一个空间里。我们就能够通过向量同时理解图片和文本了。

 复制代码

```
1 
2
3 
```

注：img 标签里的 alt 和 title 字段，提供了对图片的文本描述。

图片的零样本分类

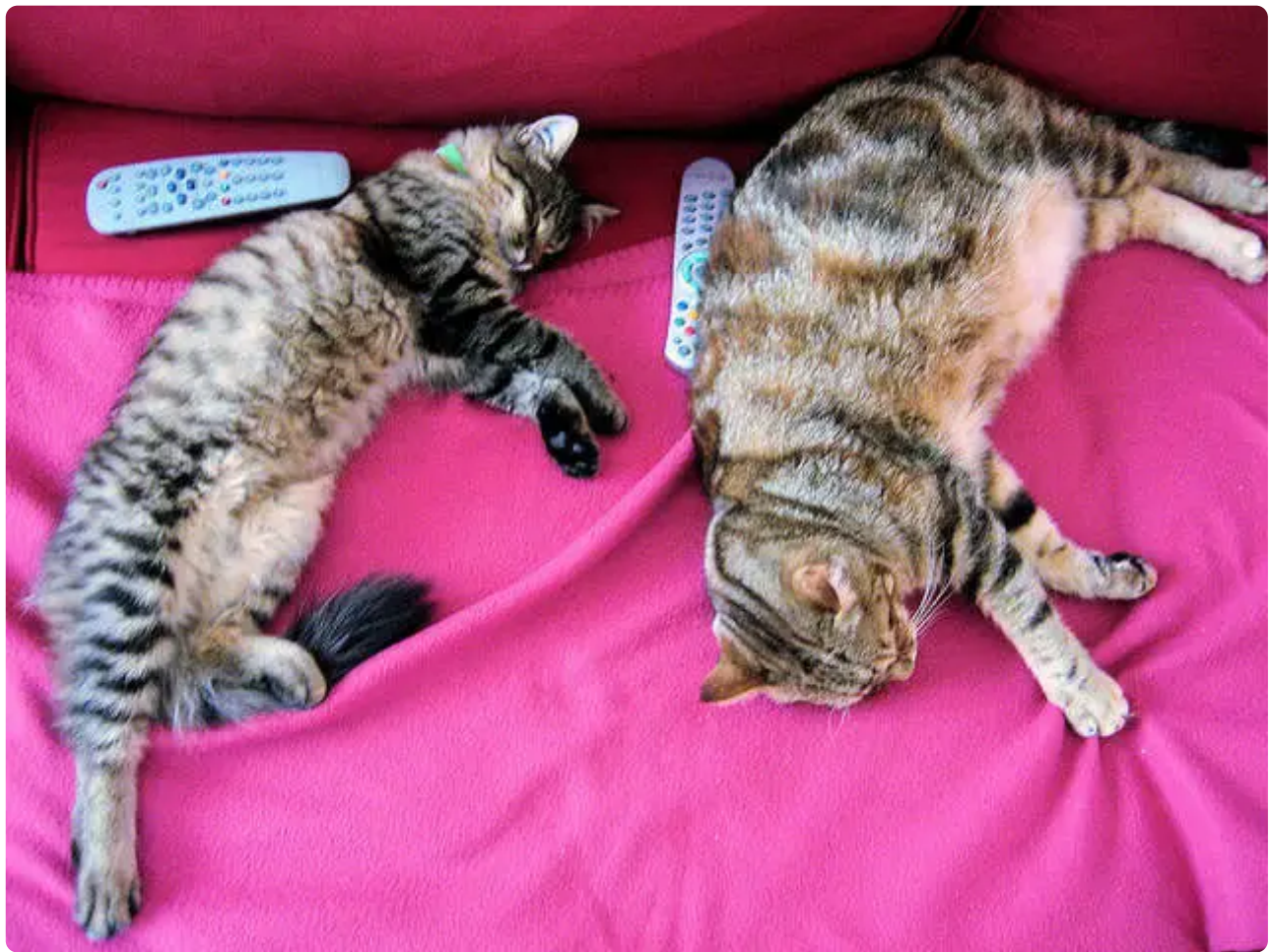
理解了 CLIP 模型的基本思路，那么我们不妨来试一试这个模型怎么能够把文本和图片关联起来。我们刚刚介绍过的 Transformers 可以说是当今大模型领域事实上的标准，那我就还是用 Transformers 库来给你举个例子好了，你可以看一下对应的代码。

 复制代码

```
1 import torch
2 from PIL import Image
3 from IPython.display import display
4 from IPython.display import Image as IPyImage
5 from transformers import CLIPProcessor, CLIPModel
6
7 model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
8 processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
9
10 def get_image_feature(filename: str):
11     image = Image.open(filename).convert("RGB")
12     processed = processor(images=image, return_tensors="pt", padding=True, truncatio
13     with torch.no_grad():
14         image_features = model.get_image_features(pixel_values=processed["pixel_v
15     return image_features
16
17 def get_text_feature(text: str):
18     processed = processor(text=text, return_tensors="pt", padding=True, truncatio
```

```
19     with torch.no_grad():
20         text_features = model.get_text_features(processed['input_ids'])
21     return text_features
22
23 def cosine_similarity(tensor1, tensor2):
24     tensor1_normalized = tensor1 / tensor1.norm(dim=-1, keepdim=True)
25     tensor2_normalized = tensor2 / tensor2.norm(dim=-1, keepdim=True)
26     return (tensor1_normalized * tensor2_normalized).sum(dim=-1)
27
28 image_tensor = get_image_feature("./data/cat.jpg")
29
30 cat_text = "This is a cat."
31 cat_text_tensor = get_text_feature(cat_text)
32
33 dog_text = "This is a dog."
34 dog_text_tensor = get_text_feature(dog_text)
35
36 display(IPyImage(filename='./data/cat.jpg'))
37
38 print("Similarity with cat : ", cosine_similarity(image_tensor, cat_text_tensor))
39 print("Similarity with dog : ", cosine_similarity(image_tensor, dog_text_tensor))
```

输出结果：



复制代码

```
1 Similarity with cat : tensor([0.2482])  
2 Similarity with dog : tensor([0.2080])
```

这个代码并不复杂，分成了这样几个步骤。

1. 我们先是通过 Transformers 库的 CLIPModel 和 CLIPProcessor，加载了 clip-vit-base-patch32 这个模型，用来处理我们的图片和文本信息。
2. 在 get_image_features 方法里，我们做了两件事情。

首先，我们通过刚才拿到的 CLIPProcessor 对图片做预处理，变成一系列的数值特征表示的向量。这个预处理的过程，其实就是把原始的图片，变成一个个像素的 RGB 值；然后统一图片的尺寸，以及对于不规则的图片截取中间正方形的部分，最后做一下数值的归一化。具体的操作步骤，已经封装在 CLIPProcessor 里了，你可以不用关心。

然后，我们再通过 CLIPModel，把上面的数值向量，推断成一个表达了图片含义的张量 (Tensor)。这里，你就把它当成是一个向量就好了。

3. 同样的，get_text_features 也是类似的，先把对应的文本通过 CLIPProcessor 转换成 Token，然后再通过模型推断出表示文本的张量。
4. 然后，我们定义了一个 cosine_similarity 函数，用来计算两个张量之间的余弦相似度。
5. 最后，我们就可以利用上面的这些函数，来计算图片和文本之间的相似度了。我们拿了一张程序员们最喜欢的猫咪照片，和 “This is a cat.” 以及 “This is a dog.” 的文本做比较。可以看到，结果的确是猫咪照片和 “This is a cat.” 的相似度要更高一些。

我们可以再多拿一些文本来进行比较。图片里面，实际是 2 只猫咪在沙发上，那么我们分别试试 “There are two cats.”、“This is a couch.” 以及一个完全不相关的 “This is a truck.”，看看效果怎么样。

 复制代码

```
1 two_cats_text = "There are two cats."
2 two_cats_text_tensor = get_text_feature(two_cats_text)
3
4 truck_text = "This is a truck."
5 truck_text_tensor = get_text_feature(truck_text)
6
7 couch_text = "This is a couch."
8 couch_text_tensor = get_text_feature(couch_text)
9
10 print("Similarity with cat : ", cosine_similarity(image_tensor, cat_text_tensor))
11 print("Similarity with dog : ", cosine_similarity(image_tensor, dog_text_tensor))
12 print("Similarity with two cats : ", cosine_similarity(image_tensor, two_cats_text_tensor))
13 print("Similarity with truck : ", cosine_similarity(image_tensor, truck_text_tensor))
14 print("Similarity with couch : ", cosine_similarity(image_tensor, couch_text_tensor))
```

输出结果：

 复制代码

```
1 Similarity with cat : tensor([0.2482])
2 Similarity with dog : tensor([0.2080])
3 Similarity with two cats : tensor([0.2723])
4 Similarity with truck : tensor([0.1814])
```

```
5 Similarity with couch : tensor([0.2376])
```

可以看到，“There are two cats.” 的相似度最高，因为图里有沙发，所以 “This is a couch.” 的相似度也要高于 “This is a dog.”。而 Dog 好歹和 Cat 同属于宠物，相似度也比完全不相关的 Truck 要高一些。可以看到，CLIP 模型对图片和文本的语义理解是非常到位的。

看到这里，你有没有觉得这和我们课程一开始的文本零样本分类很像？的确，CLIP 模型的一个非常重要的用途就是零样本分类。在 CLIP 这样的模型出现之前，图像识别已经是一个准确率非常高的领域了。通过 RESNET 架构的卷积神经网络，在 ImageNet 这样的大数据集上，已经能够做到 90% 以上的准确率了。

但是这些模型都有一个缺陷，就是它们都是基于监督学习的方式来进行分类的。这意味着两点，一个是**所有的分类需要预先定义好**，比如 ImageNet 就是预先定义好了 1000 个分类。另一个是**数据必须标注**，我们在训练模型之前，要给用来训练的图片标注好属于什么类。

这带来一个问题，就是如果我们需要增加一个分类，就要重新训练一个模型。比如我们发现数据里面没有标注“沙发”，为了能够识别出沙发，就得标注一堆数据，同时需要重新训练模型来调整模型参数的权重，需要花费很多时间。

但是，在 CLIP 这样的模型里，并不需要这样做。因为对应的文本信息，是从海量图片自带的文本信息里来的。并且因为在学习的过程中，模型也学习到了文本之间的关联，所以如果要对一张图片在多个类别中进行分类，只需要简单地列出分类的文本名称，然后每一个都和图片算一下向量表示之间的乘积，再通过 Softmax 算法做一下多分类的判别就好了。

下面就是这样一段示例代码：

```
1 from PIL import Image
2
3 from transformers import CLIPProcessor, CLIPModel
4
5 model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
6 processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

 复制代码

```
7
8 image_file = "./data/cat.jpg"
9 image = Image.open(image_file)
10
11 categories = ["cat", "dog", "truck", "couch"]
12 categories_text = list(map(lambda x: f"a photo of a {x}", categories))
13 inputs = processor(text=categories_text, images=image, return_tensors="pt", paddi
14
15 outputs = model(**inputs)
16 logits_per_image = outputs.logits_per_image
17 probs = logits_per_image.softmax(dim=1)
18
19 for i in range(len(categories)):
20     print(f"{categories[i]}\t{probs[0][i].item():.2%}")
```

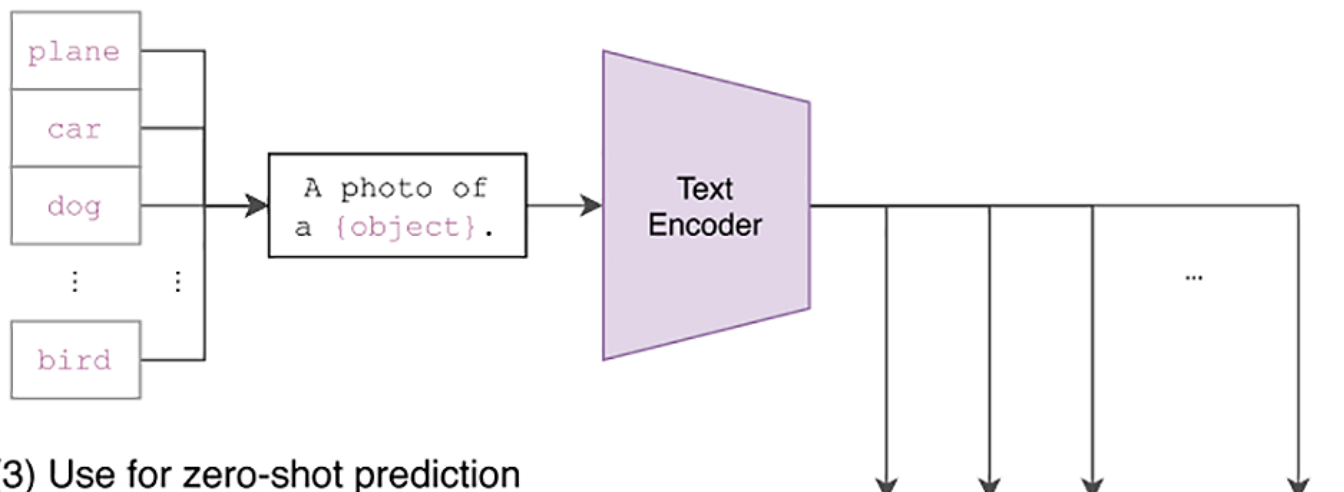
输出结果：

 复制代码

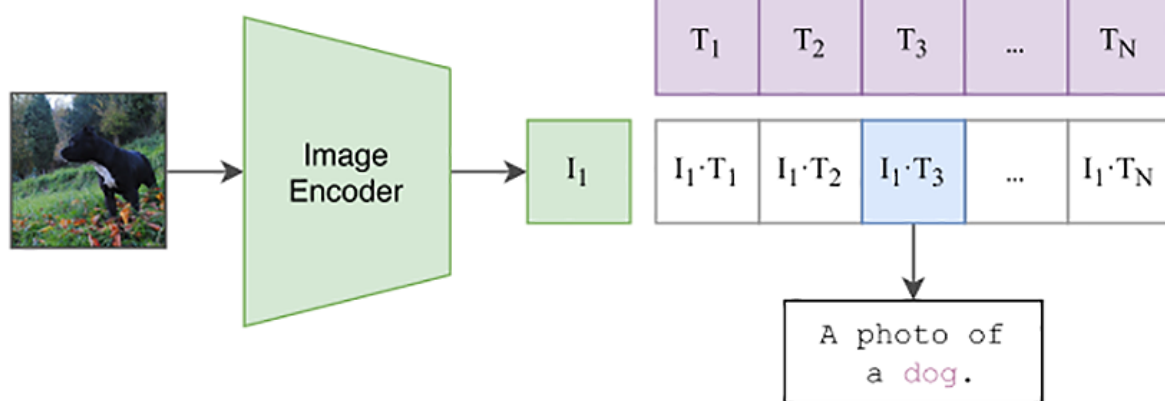
```
1 cat 74.51%
2 dog 0.39%
3 truck 0.04%
4 couch 25.07%
```

代码非常简单，我们还是先加载 model 和 processor。不过这一次，我们不再是通过计算余弦相似度来进行分类了。而是直接通过一个分类的名称，用 softmax 算法来计算图片应该分类到具体某一个类的名称的概率。在这里，我们给所有名称都加上了一个 “a photo of a ” 的前缀。这是为了让文本数据更接近 CLIP 模型拿来训练的输入数据，因为大部分采集到的图片相关的 alt 和 title 信息都不大可能会是一个单词，而是一句完整的描述。

(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



CLIP模型的论文中，关于零样本分类的示意图

我们把图片和文本都传入到 Processor，它会进行数据预处理。然后直接把这个 inputs 塞给 Model，就可以拿到输出结果了。输出结果的 logits_per_image 字段就是每一段文本和我们分类的图片在计算完内积之后的结果。我们只要再把这个结果通过 Softmax 计算一下，就能得到图片属于各个分类的概率。

从我们上面运行的结果可以看到，结果还是非常准确的，模型判断有 75% 的概率是一只猫，25% 的概率是沙发。这的确也是图片中实际有的元素，而且从图片来看，猫才是图片里的主角。

你可以自己找一些的图片，定义一些自己的分类，来看看分类效果如何。不过需要注意，CLIP 是用英文文本进行预训练的，分类的名字你也需要用英文。

通过 CLIP 进行目标检测

除了能够实现零样本的图像分类之外，我们也可以将它应用到零样本下的目标检测中。目标检测其实就是是在图像中框出特定区域，然后对这个区域内的图像内容进行分类。因此，我们同样可以用 CLIP 来实现目标检测任务。

事实上，Google 就基于 CLIP，开发了 OWL-ViT 这个模型来做零样本的目标检测，我们可以直接使用 [上一讲](#)学过的 Pipeline 来试一试它是怎么帮助我们做目标检测的。

目标检测：

 复制代码

```
1 from transformers import pipeline
2
3 detector = pipeline(model="google/owlvit-base-patch32", task="zero-shot-object-de
4 detected = detector(
5     "./data/cat.jpg",
6     candidate_labels=["cat", "dog", "truck", "couch", "remote"],
7 )
8
9 print(detected)
```

输出结果：

 复制代码

```
1 [{ 'score': 0.2868116796016693, 'label': 'cat', 'box': { 'xmin': 324, 'ymin': 20, ' '}
```

可以看到一旦用上 Pipeline，代码就变得特别简单了。我们先定义了一下 model 和 task，然后输入了我们用来检测的图片，以及提供的类别就完事了。从打印出来的结果中可以看到，里面包含了模型检测出来的所有物品的边框位置。这一次，我们还特地增加了一个 remote，也就是遥控器的类别，看看这样的小物体模型是不是也能识别出来。

接下来，我们就把边框标注到图片上，看看检测的结果是否准确。

首先，我们需要安装一下 OpenCV。

 复制代码

```
1 pip install opencv-python
```

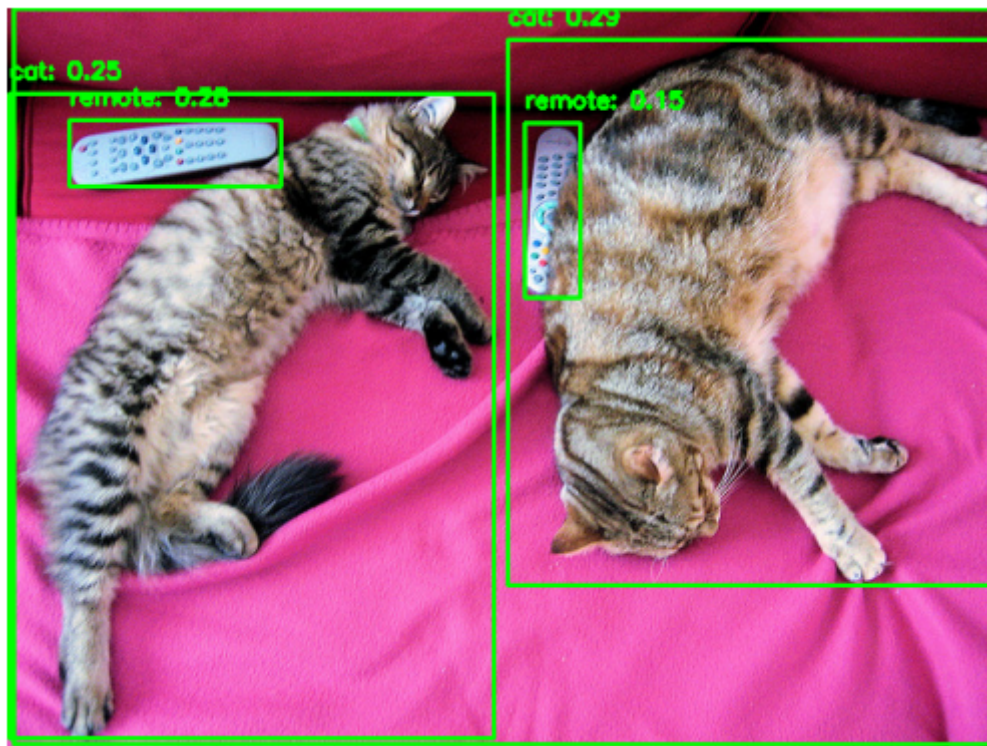
后面的代码也很简单，就是遍历一下上面检测拿到的结果，然后通过 OpenCV 把边框绘制到图片上就好了。

输出目标检测结果：

 复制代码

```
1 import cv2
2 from matplotlib import pyplot as plt
3
4 # Read the image
5 image_path = "./data/cat.jpg"
6 image = cv2.imread(image_path)
7
8 # Convert the image from BGR to RGB format
9 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
10
11 # Draw the bounding box and label for each detected object
12 for detection in detected:
13     box = detection['box']
14     label = detection['label']
15     score = detection['score']
16
17     # Draw the bounding box and label on the image
18     xmin, ymin, xmax, ymax = box['xmin'], box['ymin'], box['xmax'], box['ymax']
19     cv2.rectangle(image, (xmin, ymin), (xmax, ymax), (0, 255, 0), 2)
20     cv2.putText(image, f"{label}: {score:.2f}", (xmin, ymin - 10), cv2.FONT_HERSHEY
21
22 # Display the image in Jupyter Notebook
23 plt.imshow(image)
24 plt.axis('off')
25 plt.show()
```

输出结果：



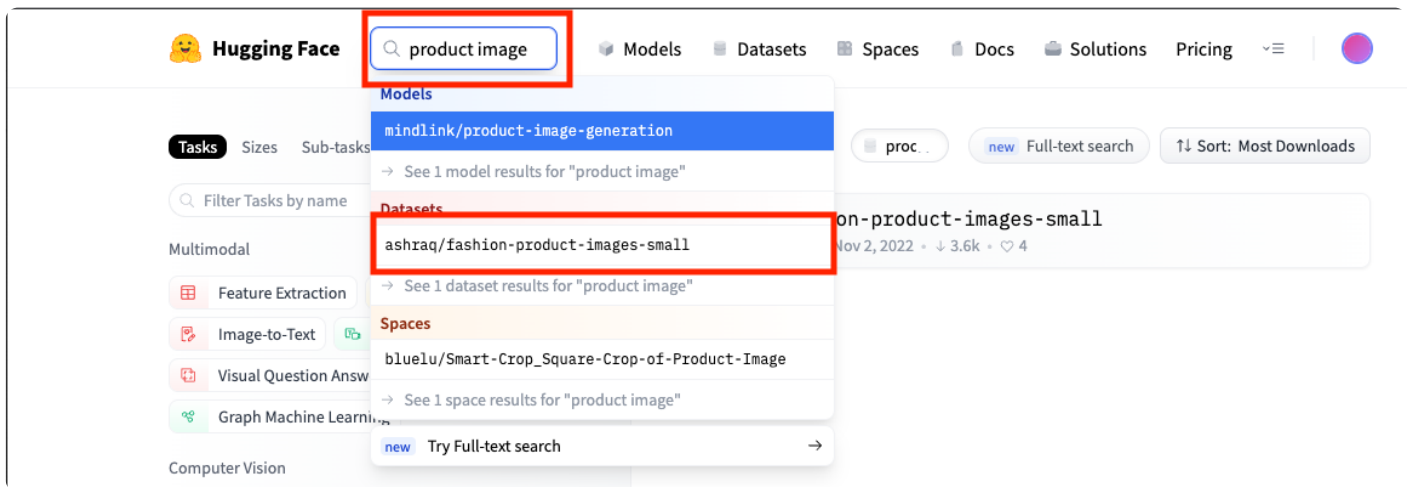
从最后的输出结果来看，无论是猫咪、遥控器还是沙发，都被准确地框选出来了。

商品搜索与以图搜图

CLIP 模型能把文本和图片都变成同一个空间里面的向量。而且，文本和图片之间还有关联，这就让我们想到了 [第 9 讲](#) 学过的内容。我们是不是可以利用这个向量来进行语义检索，实现搜索图片的功能？答案当然是可以的，其实这也是 CLIP 的一个常用功能。我们接下来就要通过代码来演示这个搜索的用法。

要演示商品搜索功能，我们要先找到一个数据集。这一次，我们需要的数据是图片，这我们就没办法直接通过 ChatGPT 来造了。不过，正好我们可以学习 HuggingFace 提供的 [Dataset 模块](#)。

所有的机器学习问题都需要有一套数据，我们需要通过数据来训练、验证和测试模型。所以作为最大的开源机器学习社区，HuggingFace 就提供了这样一个模块，让开发人员可以把他们的数据集分享出来。并且这些数据集，都可以通过 datasets 库的 `load_dataset` 方法加载到内存里面来。



在搜索栏里通过关键字查找我们想要的商品图片数据集

我们想找一些商品图片，那么就可以在 HuggingFace 的搜索栏里输入 product image。然后点击 Datasets 下找到的数据集，进入数据集的详情页。可以看到，这个叫做 [ecommece_products_clip](#) 的数据集里，的确每一条记录都有商品图片，那拿来作我们的图片搜索演示再合适不过了。

Datasets:  **rajuptvs/ecommerce_products_clip** 

 like 0

License:  mit

 **Dataset card**

 Files

 Community

Dataset Preview

Size: 48.3 MB

 API

 Go to dataset viewer

image (image)	Product_name (string)	Price (string)	colors (string)	Pattern (string)	Description (string)	Other Details (string)
	"Men Slim Fit..."	"₹799"	"Blue"	"Checkered"	"Parx Medium Blue Shirt"	"unknown"
	"Men Slim Fit Floral..."	"₹791"	"Dark Green"	"Floral Print"	"unknown"	"unknown"
	"Men Slim Fit Washed..."	"₹398"	"Dark Green"	"Washed"	"This is a best Casual wear Shirt..."	"unknown"
	"Men Regular Fit Solid..."	"₹339"	"Grey"	"Solid"	"This shirts is one of the top selling..."	"formal shirts 1"

End of preview (truncated to 100 rows)

README.md exists but content is empty. Use the *Edit dataset card* button to edit it.

加载数据集非常简单，我们只需要调用一下 `load_dataset` 方法，并且把数据集的名字作为参数就可以了。对于拿到的数据集，你可以看到里面一共有 1913 条数据，并且列出了所有 feature 的名字。

 复制代码

```
1 from datasets import load_dataset
```

```
2
3 dataset = load_dataset("rajuptvs/ecommerce_products_clip")
4 dataset
```

输出结果：

 复制代码

```
1 DatasetDict({
2     train: Dataset({
3         features: ['image', 'Product_name', 'Price', 'colors', 'Pattern', 'Descri
4         num_rows: 1913
5     })
6 })
```

数据集一般都会预先分片，分成**训练集 (train)**、**验证集 (validation)** 和**测试集 (test)** 三种。我们这里不是做机器学习训练，而是演示一下通过 CLIP 模型做搜索，所以我们选用了数据最多的 train 这个数据分片。我们通过 Matplotlib 这个库，显示了一下前 10 个商品的图片，确认数据和我们想的是一样的。

 复制代码

```
1 import matplotlib.pyplot as plt
2
3 training_split = dataset["train"]
4
5 def display_images(images):
6     fig, axes = plt.subplots(2, 5, figsize=(15, 6))
7     axes = axes.ravel()
8
9     for idx, img in enumerate(images):
10         axes[idx].imshow(img)
11         axes[idx].axis('off')
12
13     plt.subplots_adjust(wspace=0.2, hspace=0.2)
14     plt.show()
15
16 images = [example["image"] for example in training_split.select(range(10))]
17 display_images(images)
```

输出结果：



有了数据集，我们要做的第一件事情，就是通过 CLIP 模型把所有的图片都转换成向量并且记录下来。获取图片向量的方法和我们上面做零样本分类类似，我们加载了 CLIPModel 和 CLIPProcessor，通过 get_image_features 函数拿到向量，再通过 add_image_feature 函数把这些向量加入到 features 特征里面。

我们一条记录一条记录地来处理训练集里面的图片特征，并且把处理完成的特征也加入到数据集的 features 属性里面去。

 复制代码

```
1 import torch
2 import torchvision.transforms as transforms
3 from PIL import Image
4 from datasets import load_dataset
5 from transformers import CLIPProcessor, CLIPModel
6
7 device = "cuda" if torch.cuda.is_available() else "cpu"
8 model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
9 processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
10
11 def get_image_features(image):
12     with torch.no_grad():
13         inputs = processor(images=[image], return_tensors="pt", padding=True)
14         inputs.to(device)
15         features = model.get_image_features(**inputs)
16     return features.cpu().numpy()
```

```
17 def add_image_features(example):
18     example["features"] = get_image_features(example["image"])
19     return example
20
21 # Apply the function to the training_split
22 training_split = training_split.map(add_image_features)
23
```

有了处理好的向量，问题就好办了。我们可以仿照 [第 9 讲](#) 的办法，把这些向量都放到 Faiss 的索引里面去。

 复制代码

```
1 import numpy as np
2 import faiss
3
4 features = [example["features"] for example in training_split]
5 features_matrix = np.vstack(features)
6
7 dimension = features_matrix.shape[1]
8
9 index = faiss.IndexFlatL2(dimension)
10 index.add(features_matrix.astype('float32'))
```

有了这个索引，我们就可以通过余弦相似度来搜索图片了。我们通过下面四个步骤来完成这个用文字搜索图片的功能。

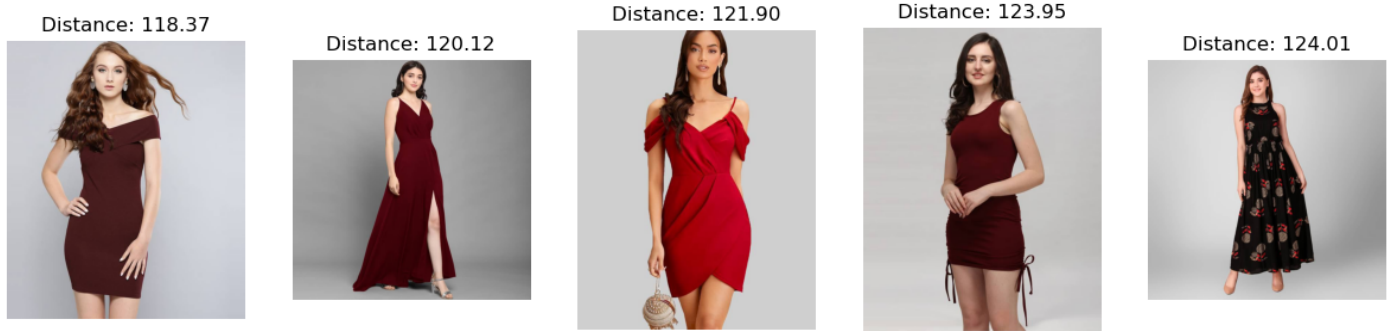
1. 首先 `get_text_features` 这个函数会通过 `CLIPModel` 和 `CLIPProcessor` 拿到一段文本输入的向量。
2. 其次是 `search` 函数。它接收一段搜索文本，然后将文本通过 `get_text_features` 转换成向量，去 Faiss 里面搜索对应的向量索引。然后通过这个索引重新从 `training_split` 里面找到对应的图片，加入到返回结果里面去。
3. 然后我们就以 `A red dress` 作为搜索词，调用 `search` 函数拿到搜索结果。
4. 最后，我们通过 `display_search_results` 这个函数，将搜索到的图片以及在 Faiss 索引中的距离展示出来。

上面这四个步骤，其实在之前的课程中都我们都讲过。我们通过这些方法的组合，就实现了一个通过关键词搜索商品图片的功能。而从搜索结果中可以看到，排名靠前的的确都是红色的裙子。

 复制代码

```
1 def get_text_features(text):
2     with torch.no_grad():
3         inputs = processor(text=[text], return_tensors="pt", padding=True)
4         inputs.to(device)
5         features = model.get_text_features(**inputs)
6     return features.cpu().numpy()
7
8 def search(query_text, top_k=5):
9     # Get the text feature vector for the input query
10    text_features = get_text_features(query_text)
11
12    # Perform a search using the FAISS index
13    distances, indices = index.search(text_features.astype("float32"), top_k)
14
15    # Get the corresponding images and distances
16    results = [
17        {"image": training_split[i]["image"], "distance": distances[0][j]}
18        for j, i in enumerate(indices[0])
19    ]
20
21    return results
22
23 query_text = "A red dress"
24 results = search(query_text)
25
26 # Display the search results
27 def display_search_results(results):
28     fig, axes = plt.subplots(1, len(results), figsize=(15, 5))
29     axes = axes.ravel()
30
31     for idx, result in enumerate(results):
32         axes[idx].imshow(result["image"])
33         axes[idx].set_title(f"Distance: {result['distance']:.2f}")
34         axes[idx].axis('off')
35
36     plt.subplots_adjust(wspace=0.2, hspace=0.2)
37     plt.show()
38
39 display_search_results(results)
```

输出结果：



有了通过文本搜索商品，相信你也知道如何以图搜图了。我们只需要把 `get_text_features` 换成一个 `get_image_features` 就能做到这一点。我也把对应的代码放在下面。

 复制代码

```
1 def get_image_features(image_path):
2     # Load the image from the file
3     image = Image.open(image_path).convert("RGB")
4
5     with torch.no_grad():
6         inputs = processor(images=[image], return_tensors="pt", padding=True)
7         inputs.to(device)
8         features = model.get_image_features(**inputs)
9     return features.cpu().numpy()
10
11 def search(image_path, top_k=5):
12     # Get the image feature vector for the input image
13     image_features = get_image_features(image_path)
14
15     # Perform a search using the FAISS index
16     distances, indices = index.search(image_features.astype("float32"), top_k)
17
18     # Get the corresponding images and distances
19     results = [
20         {"image": training_split[i.item()]["image"], "distance": distances[0][j]}
21         for j, i in enumerate(indices[0])
22     ]
23
24     return results
25
26 image_path = "./data/shirt.png"
27 results = search(image_path)
28
29 display(IPyImage(filename=image_path, width=300, height=200))
```

```
30 display_search_results(results)
31
```



用来搜索的衬衫图片

输出结果：



从搜索结果可以看到，尽管用来搜索的衬衫图片的视角和风格与商品库里面的图片完全不同，但是搜索到的图片也都是有蓝色元素的衬衫，由此可见，CLIP 模型对于语义的捕捉还是非常准确的。

小结

好了，这一讲到这里就结束了，最后我们一起来总结复习一下。

这一讲，我为你介绍了 OpenAI 开源的 CLIP 模型。这个模型是通过互联网上的海量图片数据，以及图片对应的 img 标签里面的 alt 和 title 字段信息训练出来的。这个模型无需额外的

标注，就能将图片和文本映射到同一个向量空间，让我们能把文本和图片关联起来。

通过 CLIP 模型，我们可以对任意物品名称进行零样本分类。进一步地，我们还能进行零样本的目标检测。而文本和图片在同一个向量空间的这个特性，也能够让我们直接利用这个模型进一步优化我们的商品搜索功能。我们可以拿文本的向量，通过找到余弦距离最近的商品图片来优化搜索的召回过程。我们也能直接拿图片向量，实现以图搜图这样的功能。

CLIP 这样的多模态模型，进一步拓展了我们 AI 的能力。我们现在写几个提示语，就能让 AI 拥有绘画的能力，这一点也可以认为是发端于此的。而在接下来的几讲里面，我们就要看看应该怎么使用 AI 来画画了。

思考题

你能试一试，通过 Pipeline 来实现我们今天介绍的图片零样本分类吗？进行零样本分类的时候，你选取了哪一个模型呢？欢迎你在评论区和我交流讨论，也欢迎你把这一讲分享给需要的朋友，我们下一讲再见！

推荐阅读

如果你想要对计算机视觉的深度学习有一个快速地了解，那么 Pinecone 提供的这份 [🔗 Embedding Methods for Image Search](#) 是一份很好的教程。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

精选留言 (5)



pomyin

2023-05-09 来自浙江

商品搜索与以图搜图部分，其中search函数部分代码有误（`training_split[i]["image"]`），我查了huggingface的Dataset数据类型，应该改成：`training_split.select([i])["image"]`。我的python 3.7，transformers是4.28.1。

作者回复: 可能和环境版本有关? Python 3.10 下应该是OK的, 所有Notebook我都在本地以及Colab上运行过。



peter

2023-05-03 来自北京

Q1: 笔记本, win10, 安装了Anaconda, 这个环境可以吗? 我安装了Pycharm。应该是Pycharm用来编写代码, Anaconda提供底层支持, 是这样吗?

Q2: 用某个人的声音来播音或者阅读一段文字, 有成熟的方案吗?

比如, 想用特朗普的声音来播报一段新闻, 或者读一篇文章, 是否有成熟的方案? 开源或商用的都可以。

作者回复: Anaconda只是做了Python的环境和包管理。PyCharm是对应IDE。提供Python支持的是Anaconda安装了的Python解释器和第三方包。

22讲的最后推荐了voice cloning的开源项目呀。

Azure的TTS也支持自定义模型。



Toni

2023-04-29 来自瑞士

调用了几个模型对图片进行零样本分类, 试验图像使用的是本课中的'两只猫咪', 为便于模型间的比较, 取完全相同的分类参数: `candidate_labels=["cat", "dog", "truck", "couch", "remote"]`。

模型1: `task="zero-shot-image-classification", model="openai/clip-vit-large-patch14"`

模型2: `task="zero-shot-image-classification", model="laion/CLIP-ViT-H-14-laion2B-s32B-b79K"`

模型3: `task="zero-shot-classification", model="typeform/distilbert-base-uncased-mnli"`

模型4: `task="zero-shot-classification", model="MoritzLaurer/mDeBERTa-v3-base-mnli-xnli"`

下面是各个模型运算的结果:

model	cat	dog	truck	couch	remote
1	15.03%	0.02%	0.01%	78.11%	6.82%

2	15.04%	0.00%	0.00%	84.95%	0.01%
3	13.22%	11.75%	18.13%	36.99%	19.90%
4	18.52%	16.02%	13.60%	24.22%	27.64%

这几个模型的结果都不如 `task="zero-shot-object-detection", model="google/owlvit-base-patch32"`

谷歌这个模型的优点是从图中分辨出两只猫和两个遥控器。

最后是一个中文模型, `model_id="lyua1225/clip-huge-zh-75k-steps-bs4096"`

https://huggingface.co/models?pipeline_tag=zero-shot-image-classification&language=zh&sort=downloads

```
["猫", "狗", "拖车", "长沙发", "遥控器"]  
[0.996  0.    0.    0.004  0. ]
```

没有识别出遥控器。

作者回复: owlvit 的确是现在的 state-of-the-art, 当然还有一个因素是选用的图片本身就是这个模型拿来作为例子的图片

客观评价可以多拿点图片看看



Steven

2023-04-28 来自辽宁

目标检测部分, 我的 transformers 版本是 4.24.0, 文中代码执行异常, 修改成下面的代码得到结果:

```
detected = detector("./data/cat.jpg",  
    text_queries=["cat", "dog", "truck", "couch", "remote"])[0]
```

作者回复: 👍



Santiago

2023-04-28 来自山西

欢度五一

